

High-Throughput Multi-GPU Approximate Nearest Neighbor Search with Direct Candidate Reranking

Minh Phuc Nguyen

Department ECBE

Union College

Schenectady, NY 12308, USA

nguyenp@union.edu

Abstract—Approximate Nearest Neighbor (ANN) is an indispensable component of modern retrieval systems, artificial intelligence, recommendation algorithms, and others. However, processing large, high-dimensional datasets requires balancing the trade-off among absolute accuracy, query throughput, and memory consumption. This study builds a multi-GPU ANN evaluation and optimization system based on the NVIDIA RAPIDS cuVS library. The experiments are conducted on a public dataset released by OpenAI, containing 5 million embedding vectors with 1536 dimensions each, and are run in a personal computing environment with 2 RTX 5070 Ti GPUs as the primary search devices. To compensate for the accuracy loss caused by compression and approximate search, we develop a 2-stage search program in which the search algorithm from the cuVS library returns a result set larger than the required number of results, and a CUDA reranking algorithm produces the final candidates. Experimental results show that IVF-PQ achieves $\approx 195k$ queries per second at a Recall@10 of 0.8752 and reaches its highest Recall@10 of 0.9930 at $\approx 18k$ queries per second. For CAGRA, the highest Recall@10 without reranking is only 0.8323; reranking the candidate set increases this value to 0.9616 at $\approx 87k$ queries per second. Another high-throughput CAGRA configuration reaches $\approx 158k$ queries per second with a Recall@10 of 0.9471. These results show that exact reranking over the candidate set can provide multiple flexible operating points between throughput and accuracy for multi-GPU ANN systems.

Index Terms—approximate nearest neighbor, multi-GPU, vector search, IVF-PQ, CAGRA, CUDA reranking

I. INTRODUCTION

The rapid development of deep learning models has required an enormous amount of data for training. Unstructured data such as text, images, and video have become important inputs for AI, creating a need to represent data as high-dimensional embedding vectors. The applications are broad, including information retrieval, computer vision, natural language processing, recommendation systems, and others [1], [2].

Given a dataset $D = \{x_1, x_2, \dots, x_N\}$, where $x_i \in \mathbb{R}^d$, and a query vector q , the k -nearest-neighbor search problem requires finding the k vectors in D with the smallest distances to q . As the dataset becomes extremely large with millions of vectors and thousands of dimensions, scanning the entire dataset for every query becomes inefficient and cannot meet system latency and throughput requirements. To deal with that problem, practical systems commonly employ approximate nearest-neighbor search, which trades some retrieval accuracy, measured by Recall@ k , for higher query throughput and lower

latency. For ANN search over high-dimensional vectors and datasets, two influential families of methods are graph-based methods and methods based on partitioning combined with quantization.

HNSW [3] is a representative graph-based approach. The algorithm creates a neighborhood graph with multiple layers to rapidly locate the search region before performing detailed traversal at the base layers. HNSW is broadly used in production environment; however, it was originally developed and is still commonly deployed as a CPU-oriented method. Because graph traversal is data-dependent, memory accesses are non-contiguous, and the candidate set must be maintained dynamically, directly mapping HNSW to the SIMT architecture of GPUs presents many difficulties. To address the weaknesses of HNSW, CAGRA was designed directly for GPU architectures to exploit their strengths. CAGRA uses a graph with a fixed out-degree, distributes work uniformly among threads, and optimizes both construction and search to exploit the parallel processing capability of GPUs [1].

For the partitioning-based approach, IVF-PQ is a representative method that combines an inverted file (IVF) with product quantization (PQ) [4]. IVF assigns each vector to a partition determined by a coarse quantizer and examines only a number of partitions whose centroids are closest to the query. Within each partition, PQ divides the residual vector into multiple subvectors and quantizes each part independently, allowing every vector to be represented by a short code and its distance to be estimated quickly through lookup tables. Subsequently, Johnson et al. optimized compressed-domain distance computation and top- k selection on GPUs, thereby showing that this approach can achieve high performance at the scale of billions of vectors [2]. However, IVF-PQ may miss neighbors located outside the probed partitions, while quantization error can also change the ordering of candidates. Therefore, an effective approach is to generate a candidate set larger than k and then rerank this set using higher-precision information [5]. In this study, we apply this principle but recompute the squared Euclidean distance directly over the original vectors using GPUs, rather than continuing to estimate distances from quantized codes.

This paper focuses on a practical question: how can high throughput be achieved while maintaining reliable Recall@10 when searching a 5-million-vector, 1536-dimensional OpenAI

embedding dataset using two commodity GPUs with limited memory? We investigate two index families in NVIDIA RAPIDS cuVS: sharded IVF-PQ and compressed CAGRA. The approximate search will combine generating a candidate set larger than the required result set and reranking to select the ten best results with a C++/CUDA module.

The planned main contributions of this paper are summarized as follows:

- Build an end-to-end multi-GPU ANN evaluation pipeline on a dataset of 5 million vectors, compute Recall@10, and examine both batch throughput and single-query latency.
- Design and implement a multi-GPU C++/CUDA reranking module.
- Conduct a systematic parameter sweep to characterize the trade-off frontier between throughput and Recall@10.

II. SYSTEM DESIGN AND RESEARCH METHODOLOGY

A. System Overview

For a query q , the approximate index first returns a candidate set:

$$C(q) = \{x_1, x_2, \dots, x_r\}$$

where r is greater than the required number of returned results k . The squared Euclidean distance between the query and each candidate is then recomputed as

$$d(q, x_i) = \sum_{j=1}^d (q_j - x_{i,j})^2,$$

and the $k = 10$ candidates with the smallest distances are selected as the final results.

The ANN index and the data used for reranking are independently sharded across two GPUs. The IVF-PQ or CAGRA index uses the SHARDED mode of cuVS to search the data partitions and merge local results into a global candidate list. In the reranking module, the original vector matrix is divided into two contiguous row segments, with each segment resident on one GPU. The complete processing flow, from approximate search over the sharded indexes to result merging after reranking, is illustrated in Fig. 1.

B. Candidate Generation Using IVF-PQ and CAGRA

For IVF-PQ, the index is divided across the two GPUs. Each GPU searches its local data partition, and the local candidate lists are then merged into one global result list. Within each partition, a coarse quantizer divides the vectors into n_{lists} inverted lists. At query time, n_{probes} specifies how many of the lists closest to the query are searched. Product quantization compresses each vector into pq_{dim} code components, with pq_{bits} bits used for each component. Increasing the number of probed lists generally improves candidate coverage but requires more vectors to be examined, while increasing the size of the PQ representation preserves more distance information at the cost of additional memory and computation [4], [5].

For CAGRA, the index is a graph with a fixed number of outgoing edges per vertex. The intermediate graph degree

specifies the number of neighbors per vertex in the initial graph before pruning, whereas the graph degree specifies the number retained in the final search graph. During traversal, `itopk_size` controls the number of promising intermediate candidates maintained by the search, and `search_width` controls how many graph vertices are selected for expansion in each iteration. Larger values generally explore the graph more broadly, potentially improving candidate coverage while increasing search cost. IVF-PQ and PQ have two separate roles in this configuration: IVF-PQ is used only to find approximate neighbors when constructing the initial graph, while PQ compression controls the vector representation stored in the CAGRA index. After construction, queries traverse the final graph edges rather than probing inverted lists as in the standalone IVF-PQ backend [1], [6].

C. Direct Reranking on Multiple GPUs

The reranking module is implemented in C++/CUDA and is used for both IVF-PQ and CAGRA. The data are divided into contiguous row segments, with each segment resident on one GPU. In resident mode, the query and all candidate row offsets are sent to both GPUs. Each GPU checks the row offset of every candidate; only a block on the GPU that owns the vector computes a finite distance. This implementation keeps the buffer layout identical across GPUs, but still incurs the overhead of checking candidates on a GPU that does not own them.

Each query–candidate pair is processed by one CUDA block containing 256 threads. The threads divide the 1,536 vector dimensions among themselves, convert each value to FP32, compute the squared difference, and accumulate it locally. The local sums are then reduced through shared memory to produce the final squared Euclidean distance. After the distance kernel, each GPU uses one thread per query to sequentially select the best $k = 10$ results within the shard.

Each GPU transfers at most k distances and row offsets per query back to the CPU. With two GPUs, the merging process needs to examine at most 20 elements, removes duplicated `dataset_id` values, and uses the row offset as the tie-breaking rule when distances are equal. The IVF-PQ session configuration stores both queries and reranking data in FP16 and accumulates in FP32; the CAGRA path can use FP32 queries with FP16 resident data. Therefore, reranking eliminates errors from PQ lookup tables but is not completely equivalent to computation over the original FP32 data.

D. Memory Organization and Asynchronous Execution

The original FP32 dataset requires approximately ≈ 30 GB of memory for five million 1536-dimensional vectors. If it is evenly divided across two GPUs, the reranking data alone occupy approximately ≈ 15 GB on each device, leaving insufficient space for the index and intermediate memory regions. Therefore, the system stores the reranking shards in FP16, reducing their size to approximately 7GB per GPU.

To reduce data-transfer and memory-allocation costs along the search path, the system uses pinned host memory, non-

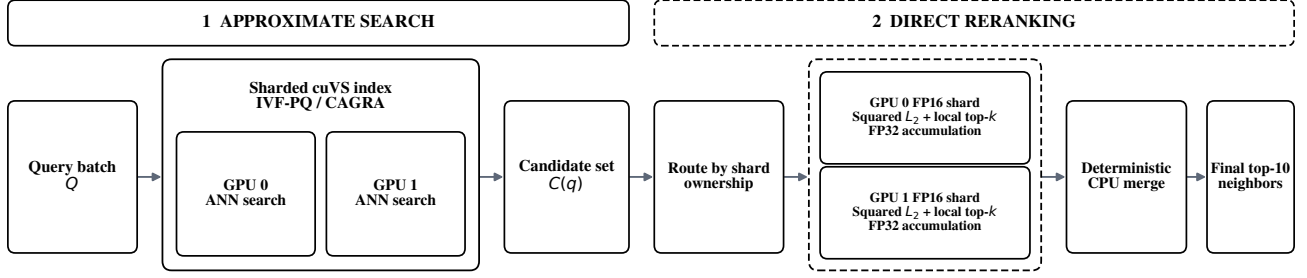


Fig. 1. Two-stage multi-GPU ANN search pipeline. A sharded cuVS index generates $r > k$ candidates using IVF-PQ or CAGRA. Candidates are routed to their resident FP16 shards for direct squared- L_2 reranking with FP32 accumulation and local top- k selection, followed by a deterministic CPU merge.

blocking CUDA streams, and memory regions that are allocated once and reused across multiple queries. Two buffers are used alternately: while one query batch is being processed on the GPU, the next batch can be prepared or transferred asynchronously.

III. EXPERIMENTAL SETUP

A. Dataset and Experimental Environment

The experiments are conducted on the `openai_large_5m` dataset [7], which contains 5 million embedding vectors with 1536 dimensions each. Ground truth is provided in the file, where each query is accompanied by an ordered list of at least 100 nearest neighbors. The first ten elements of this list are used to compute Recall@10. CAGRA results are mapped from internal row offsets to datasetID values before comparison with the ground truth.

The experimental system uses an AMD Ryzen 9 9900X CPU with 12 cores and 24 threads, 60 GiB of RAM, and two NVIDIA GeForce RTX 5070 Ti GPUs with 16 GiB of memory per GPU. In addition, the program uses multiple functions from the cuVS 26.06 library [6].

B. Configuration Space

For IVF-PQ, the study evaluates 49 configurations. The number of inverted lists, or the number of partitions created by the coarse quantizer, is selected from the set

$$n_{\text{lists}} \in \{1.024, 2.048, 4.096, 8.192\},$$

with pq_{bits} , the number of bits used to encode each component of a PQ code, set to 4 or 8; pq_{dim} , the number of subspaces into which a vector is divided during quantization, set to 384 or 768; and n_{probes} , the number of inverted lists probed for each query, varied from 8 to 128. Additional configurations are concentrated around regions likely to form the Recall-Throughput trade-off frontier; therefore, not every parameter combination is evaluated.

For CAGRA, the graph-degree and intermediate-graph-degree pairs, candidate-set size, and `search_width`, or the number of vertices expanded concurrently in each traversal iteration, are selected according to Section II-B. The sweep consists of 60 configurations without reranking and 50 configurations with reranking. Batch size, or the number of

queries processed concurrently in each internal batch, is set to 64 or 128; two representative configurations are additionally evaluated with a batch size of 256. Without reranking, `itopk_size`, or the number of best intermediate candidates maintained during graph traversal, is swept to evaluate the recall limit of direct search over the compressed index. The remaining low-level CAGRA parameters are kept in cuVS automatic mode.

C. Evaluation Metrics and Measurement Procedure

Accuracy is evaluated using Recall@10:

$$\text{Recall@10} = \frac{1}{10N_q} \sum_{i=1}^{N_q} |R_i \cap G_i|,$$

where R_i and G_i are respectively the set of ten returned results and the set of ten ground-truth neighbors for the i -th query.

Each configuration is evaluated in two modes. Offline mode processes all 1000 queries to measure batch throughput, while online mode uses a single query to examine latency when inter-query parallelism is limited. Throughput is computed as

$$\text{QPS} = \frac{N_q}{T_{\text{median}}},$$

in offline mode. For a single query, the study records median latency and p95 latency. The reported time includes approximate search, reranking, top- k selection, and result merging across two GPUs. Data loading, ground-truth generation, and index construction time are not included in QPS; index construction time is recorded separately.

IV. EXPERIMENTAL RESULTS

A. Recall-Throughput Trade-off of IVF-PQ

Figure 2 presents the results of 49 IVF-PQ configurations with direct reranking over 100 candidates. The configurations form a clear trade-off frontier: the highest throughput reaches approximately 195k queries/s at a Recall@10 of ≈ 0.875 , whereas the highest Recall@10 reaches 0.9930 at approximately 18k queries/s. This gap shows that optimizing IVF-PQ does not lead to a single best configuration, but instead produces multiple operating points according to accuracy requirements.

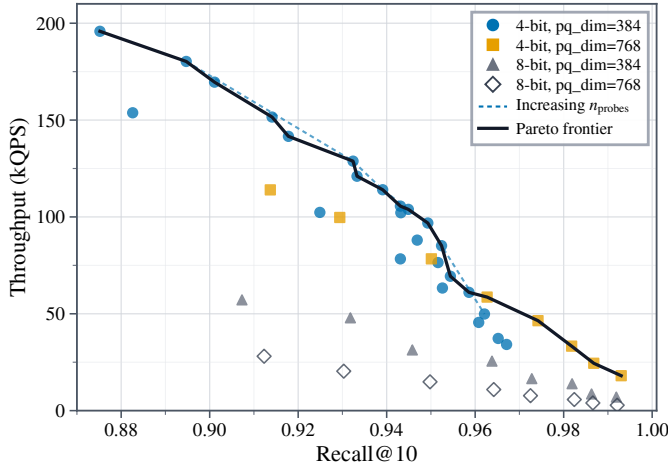


Fig. 2. Recall@10–Throughput trade-off of 49 IVF-PQ configurations, where each query generates 100 candidates before reranking. Each marker represents a configuration $(n_{\text{lists}}, pq_{\text{bits}}, pq_{\text{dim}}, n_{\text{probes}})$; the legend groups points only by pq_{bits} and pq_{dim} , whereas the other two dimensions, n_{lists} and n_{probes} , vary across points but are not encoded separately by color or marker. The dashed line connects configurations with $n_{\text{lists}} = 4096$, $pq_{\text{bits}} = 4$, and $pq_{\text{dim}} = 384$ in the direction of increasing n_{probes} ; the black line represents the Pareto frontier. The blue-circle group contains 25 points because, in addition to the eight base configurations, 17 additional configurations were examined around the region likely to extend the Pareto frontier, whereas each remaining group contains eight points.

Within the same index-build configuration, n_{probes} is the parameter that directly affects this trade-off. With $n_{\text{lists}} = 4096$, $pq_{\text{bits}} = 4$, and $pq_{\text{dim}} = 384$, increasing n_{probes} from 16 to 128 increases Recall@10 from 0,8947 to 0,9621, but reduces throughput from approximately 180,3 thousand to 49,9 thousand queries/s. Probing more inverted lists increases the likelihood of placing the correct neighbors in the candidate set, while also increasing the number of PQ codes that must be read and evaluated for each query.

All 19 points on the Pareto frontier in the experiment use $pq_{\text{bits}} = 4$. The 8-bit configurations can achieve high recall, but require larger PQ codes to be processed and do not produce any point that is simultaneously better in both recall and throughput. This result applies only to the examined workload and implementation; it does not mean that 4-bit PQ is always better than 8-bit PQ on every dataset.

The effect of pq_{dim} varies by recall region. Configurations with $pq_{\text{dim}} = 384$ dominate the high-throughput region and recall below approximately 0,96. When higher recall is required, $pq_{\text{dim}} = 768$ begins to appear on the Pareto frontier. For example, with the parameter order $(n_{\text{lists}}, pq_{\text{bits}}, pq_{\text{dim}}, n_{\text{probes}})$, configuration (2048, 4, 768, 64) achieves a Recall@10 of 0,9818 at approximately 33,3 thousand queries/s, whereas (2048, 4, 768, 128) achieves the highest recall value of 0,9930. Increasing pq_{dim} produces a more detailed PQ representation and reduces quantization error, but also increases the code size and the cost of distance computation.

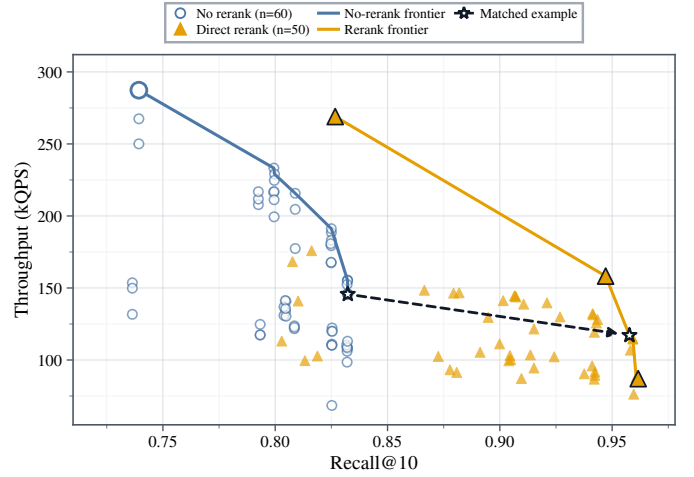


Fig. 3. Comparison of the Recall@10–Throughput spaces of 60 CAGRA configurations without reranking and 50 configurations with direct reranking. The solid lines represent the Pareto frontier of each sweep, whereas the black dashed line connects the representative configuration pair (32, 64, 24, 128, 2).

B. Impact of Direct Reranking on CAGRA

Because the two sweeps do not have completely identical parameter spaces, the overall frontier is used to represent the operating region of each method, while the direct impact of reranking is analyzed on 48 matched configuration pairs with the same graph degree, intermediate graph degree, itopk_size , batch size, and search_width .

Without reranking, the configuration with the highest throughput reaches approximately 287,3 thousand queries/s, but its Recall@10 is only 0,7393. The highest Recall@10 in this group is 0,8323, achieved at approximately 145,7 thousand queries/s. No configuration without reranking reaches a Recall@10 of 0,90 or higher, showing that errors during search and ranking on the compressed representation impose a clear accuracy limit.

When reranking is enabled, CAGRA returns between 16 and 48 candidates, after which direct distances are recomputed over the candidate vectors to select the final 10 results. With the parameter order (graph degree, intermediate graph degree, itopk_size , batch size, search width), configuration (16, 32, 32, 256, 1) achieves the highest throughput of approximately 269,0 thousand queries/s at a Recall@10 of 0,8267. Configuration (32, 64, 48, 256, 1) achieves a Recall@10 of 0,9471 while still maintaining approximately 158,2 thousand queries/s. The highest Recall@10 is 0,9616, achieved by configuration (32, 64, 24, 64, 2) at approximately 87,0 thousand queries/s.

This impact is more evident when the matched configurations are compared. Across all 48 common configuration pairs, reranking increases Recall@10 in every case, with a median increase of approximately 0,107. At the same time, 47 of the 48 pairs have lower throughput, with a median reduction of approximately 26,1%. For example, with configuration (32, 64, 24, 128, 2), search without reranking achieves a Recall@10 of 0,8323 at approximately 145,7 thousand queries/s.

TABLE I

RECALL-RECOVERY MECHANISM OF DIRECT RERANKING ON FOUR REPRESENTATIVE CAGRA CONFIGURATIONS. VALUES ARE MEDIANS OF THREE INDEPENDENT GRAPH BUILDS. A CONFIGURATION IS WRITTEN IN G/IG/R/B/W ORDER, DENOTING GRAPH DEGREE, INTERMEDIATE GRAPH DEGREE, NUMBER OF RERANKING CANDIDATES, BATCH SIZE, AND SEARCH WIDTH, RESPECTIVELY. B0 IS THE NO-RERANKING PATH WITH $\text{search_k} = 10$, B1 IS THE MATCHED NO-RERANKING PATH WITH $\text{search_k} = R$, AND RR IS THE COMPLETE RERANKING PIPELINE. HIDDEN@R IS THE RECALL ALREADY PRESENT IN THE CANDIDATE SET BUT RANKED BELOW POSITION 10; γ_{10} IS THE NORMALIZED GAP AT THE TOP-10 BOUNDARY; INVERSION IS THE PROPORTION OF CANDIDATE PAIRS WHOSE ORDER IS REVERSED RELATIVE TO THE FP32 ORACLE.

Configuration G/IG/R/B/W	Recall and Candidate Quality					Recovery Diagnostics			Performance	
	Raw R@10	Cand. R@R	Hidden @R	Rerank R@10	Oracle R@10	Recovery (%)	Median γ_{10}	Inversion (%)	kQPS B0/B1/RR	Δ QPS (%)
(32/64/24/64/2)	0.8371	0.9585	0.1214	0.9585	0.9585	100.0	0.003981	14.14	109.1/106.7/92.4	15.1
(32/64/48/128/1)	0.8371	0.9609	0.1238	0.9609	0.9609	100.0	0.003936	13.98	171.3/160.8/126.2	23.7
(24/48/48/128/2)	0.8286	0.9462	0.1172	0.9462	0.9462	100.0	0.004011	13.86	196.2/175.7/142.3	27.5
(16/32/32/128/2)	0.8058	0.9118	0.1047	0.9118	0.9118	100.0	0.003945	13.88	224.6/212.6/166.3	26.0

When 24 candidates are reranked, Recall@10 increases to 0.9577, while throughput decreases to approximately 117,1 thousand queries/s. Thus, this configuration gains 0,1254 Recall@10 points, equivalent to 12,54 percentage points, at a throughput cost of approximately 19,6%.

The issue with no-reranking configurations that have low recall does not necessarily lie in the graph-traversal process failing to find the correct neighbors, because the candidate set returned by this process may already contain the required answers, but they have not appeared in the first 10 positions. Table I shows that, whereas the Recall@10 of the initial ordering lies only in the range 0,8058–0,8371, CandidateRecall@R ranges from 0,9118 to 0,9609. Thus, a substantial fraction of the correct neighbors, corresponding to 0,1047–0,1238 recall, already appears in the complete candidate set but is ranked below position 10. At the same time, the median normalized gap at the boundary between candidates at positions 10 and 11 is only approximately 0,4%, while approximately 13,86–14,14% of candidate pairs are inverted relative to the FP32-oracle ordering. These results show that the direct source of error lies in the approximate ordering produced by CAGRA/PQ, rather than being attributable primarily to storing vectors in FP16. In fact, the reranking path, which stores data in FP16 and accumulates distances in FP32, achieves the same Recall@10 as the FP32 oracle in all four configurations. Therefore, recomputing direct distances over the candidate set and sorting candidates using more accurate distances brings the correct answers that are ranked below position 10 back into the leading result group, thereby improving recall.

C. Overall Comparison and Operating-Point Selection

To compare the two methods at equivalent quality levels, the study divides the results into three regions: Recall@10 around 0,90, corresponding to the range from 0,895 to 0,910; Recall@10 around 0,95, corresponding to the range from 0,945 to 0,960; and the very-high-recall region with Recall@10 of 0,98 or higher. Within each region, the configuration with the highest throughput for each method is selected as the representative point. This selection avoids directly comparing

configurations with overly different recall levels, while also reflecting the most efficient operating point that each method achieves under the same accuracy requirement.

The three configurations representing the recall regions around 0,90, around 0,95, and at or above 0,98 are (8192, 4, 384, 24), (4096, 4, 384, 56), and (2048, 4, 768, 64), respectively. The two configurations representing the recall regions around 0,90 and around 0,95 are (16, 32, 48, 128, 1) and (32, 64, 48, 256, 1), respectively.

Thus, in this operating region, the representative IVF-PQ configuration has approximately 17,4% higher throughput and lower median single-query latency. However, CAGRA achieves lower p95 latency.

At equivalent recall, the CAGRA configuration has approximately 63,4% higher throughput. Nevertheless, IVF-PQ still achieves lower median latency; conversely, CAGRA has lower p95 latency. This result shows that the throughput of a batched-query workload and the latency of a single query reflect two different operating modes and therefore do not necessarily rank configurations in the same order.

No CAGRA configuration with reranking reaches this recall region. This result does not mean that CAGRA is inherently unable to achieve a Recall@10 of 0,98, but only shows the observed limit within the parameter space, compressed representation, and reranking pipeline examined in this study.

TABLE II

COMPARISON OF REPRESENTATIVE OPERATING POINTS OF IVF-PQ AND CAGRA WITH DIRECT RERANKING IN THREE RECALL REGIONS. ONLINE MEDIAN/p95 REPRESENTS SINGLE-QUERY LATENCY, WHEREAS THROUGHPUT IS MEASURED ON A BATCHED-QUERY WORKLOAD.

Recall region	Method	Recall@10	Throughput (kQPS)	Online median/p95 (ms)	Build time (s)
$\approx 0,90$	IVF-PQ	0,9011	169,6	0,472/0,890	11,58
$\approx 0,90$	CAGRA + rerank	0,9065	144,4	0,610/0,614	19,41
$\approx 0,95$	IVF-PQ	0,9493	96,9	0,497/0,884	8,57
$\approx 0,95$	CAGRA + rerank	0,9471	158,2	0,604/0,716	20,22
$\geq 0,98$	IVF-PQ	0,9818	33,3	0,530/0,555	8,02
$\geq 0,98$	CAGRA + rerank			Not achieved	

Build time also creates a notable difference between the two methods. Therefore, in addition to Recall and Throughput, index-build cost should also be considered for applications that frequently update data or must rebuild the index. However, these values depend on the dataset, hardware, and implementation used, and should therefore not be interpreted as a universal advantage of IVF-PQ.

IVF-PQ is more suitable in the recall region around 0,90 when an application simultaneously prioritizes throughput, median latency, and build time. In the recall region around 0,95, CAGRA with reranking is a more suitable choice for batched-query workloads because it achieves significantly higher throughput, while IVF-PQ still has an advantage in median latency and index-build cost. When an application requires Recall@10 of 0,98 or higher, IVF-PQ is the only method that satisfies the requirement among the configurations examined.

D. Distance to the Theoretical Throughput Limit

To assess the degree of hardware utilization, the study compares the measured throughput with a *streaming ceiling* estimated from the mandatory data volume of each configuration. The system uses two RTX 5070 Ti GPUs, each with GDDR7 bandwidth of 896 GB/s, corresponding to a total peak bandwidth of 1,792 TB/s when work is distributed with perfect balance [8]. Given logical traffic of D bytes per query, the bandwidth-based throughput limit is computed as

$$QPS_{\text{stream}} = \frac{2BW_{\text{GPU}}}{D}.$$

This value is a conditional limit: it assumes balanced shards and that all logical accesses are served from GDDR7, while not fully accounting for coarse search, lookup-table construction, top- k , synchronization, and merging.

For IVF-PQ, the logical data volume per query is estimated by

$$D_{\text{IVF}} = N \frac{n_{\text{probes}}}{n_{\text{lists}}} \left(pq_{\text{dim}} \frac{pq_{\text{bits}}}{8} + 8 \right) + Rds,$$

where 8 bytes represent the ID of each vector, $R = 100$ is the number of reranking candidates, $d = 1536$, and $s = 2$ bytes for FP16. With configuration (4096, 4, 384, 32), logical traffic reaches approximately 8,12 MB/query, producing a streaming ceiling of 220,7 thousand queries/s. The measured throughput is 146,6 thousand queries/s at a Recall@10 of 0,9321, corresponding to 66,4% of the ceiling. When n_{probes} increases to 56, traffic increases to 13,98 MB/query and the ceiling decreases to 128,2 thousand queries/s. This configuration achieves 96,9 thousand queries/s at a Recall@10 of 0,9493, corresponding to approximately 75,6% of the ceiling.

For CAGRA, the amount of work is not fixed because graph traversal may terminate early depending on the query. With configuration (32, 64, 48, 256, 1), AUTO mode selects SINGLE_CTA, itopk_size is rounded up to 64, and max_iterations=0 corresponds to at most approximately 54 iterations on each shard [6]. If both shards execute the

TABLE III

COMPARISON OF MEASURED THROUGHPUT WITH THE STREAMING CEILING BASED ON LOGICAL TRAFFIC AND THE TOTAL PEAK GDDR7 BANDWIDTH OF THE TWO GPUS. CAGRA VALUES MARKED WITH * CORRESPOND TO A CONDITIONAL FULL-ITERATION PATH AND DO NOT FULLY INCLUDE THE COSTS OF CONTROL, HASH TABLES, TOP- k , AND SYNCHRONIZATION.

Configuration and Accuracy			Bandwidth Model		Measured Performance	
Method	Configuration	Recall@10	Logical traffic/query	Streaming ceiling	Measured QPS	Ratio
IVF-PQ	(4096, 4, 384, 32)	0,9321	8,12 MB	220,7k	146,6k	66,4%
IVF-PQ	(4096, 4, 384, 56)	0,9493	13,98 MB	128,2k	96,9k	75,6%
CAGRA + RR	(32, 64, 48, 256, 1)	0,9471	1,49 MB*	1.204k*	158,2k	13,1%*

full number of iterations, the number of candidate-distance evaluations is estimated as

$$N_{\text{eval}} = 2 \times 54 \times 1 \times 32 = 3456.$$

When accounting for the 384-byte compressed code, the 32-bit graph index, and 48 reranking vectors, logical traffic reaches approximately 1,49 MB/query, corresponding to a conditional streaming ceiling of approximately 1,20 million queries/s. The measured throughput is 158,2 thousand queries/s, equal to approximately 13,1% of this value. The large gap does not indicate that CAGRA uses bandwidth inefficiently, but instead shows that the streaming model does not fully represent the random memory accesses, hash-table lookup, candidate maintenance, CTA synchronization, and top- k selection of graph traversal.

Overall, the bandwidth model produces a ceiling relatively close to the IVF-PQ results, showing that the volume of PQ codes that must be scanned is the dominant component. In contrast, the CAGRA ceiling is considerably looser because graph-traversal performance is strongly affected by memory-access latency and control dependencies. Therefore, the distance to the speed-of-light limit should be interpreted according to the characteristics of each algorithm, rather than by applying a single bandwidth model to both methods.

V. CONCLUSION

This study built and evaluated a multi-GPU ANN pipeline for a dataset of 5 million 1536-dimensional embedding vectors, using IVF-PQ and compressed CAGRA combined with direct reranking over the candidate set. The results show that method selection depends substantially on the recall region and workload characteristics. IVF-PQ provides a broader operating range and is the only method to achieve Recall@10 above 0,98 within the examined configuration space. In contrast, in the Recall@10 region around 0,95, CAGRA with reranking achieves significantly higher throughput, with 158,2 thousand queries/s at a Recall@10 of 0,9471.

Direct reranking plays an important role in correcting ordering errors produced by search over the compressed representation. Across 48 matched CAGRA configuration pairs, reranking improves Recall@10 in every case, with a median increase of 0,107 and a median throughput reduction of approximately 26,1%. Candidate-set analysis shows that many

correct neighbors have already been found by graph traversal but are ranked below the first 10 positions; therefore, direct distance recomputation can improve recall without substantially expanding the approximate-search process.

The throughput-limit analysis also shows that IVF-PQ is governed primarily by the volume of PQ codes that must be scanned, whereas CAGRA performance additionally depends strongly on noncontiguous memory accesses and the control cost of graph traversal. Overall, the study results show that no single configuration is optimal for every requirement. IVF-PQ is more suitable when very high recall is prioritized, whereas CAGRA combined with reranking is an efficient choice for batched-query workloads in the medium-to-high recall region. Future studies can extend the evaluation to more datasets and GPU architectures, while dynamically selecting the candidate-set size according to recall requirements and system resource limits.

REFERENCES

- [1] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, and Y. Wang, “CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs,” in *Proc. 40th IEEE Int. Conf. Data Engineering (ICDE)*, 2024, pp. 4236–4247.
- [2] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [3] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [4] H. Jégou, M. Douze, and C. Schmid, “Product quantization for nearest neighbor search,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2011.
- [5] NVIDIA Corporation, “IVF-PQ: cuVS indexing guide,” NVIDIA cuVS Documentation. [Online]. Available: <https://docs.nvidia.com/cuvs/user-guide/api-guides/indexing-guide/ivf-pq>, 2026, accessed: Aug. 25, 2026.
- [6] —, “cuVS: Vector search and clustering on the GPU, version 26.06.00,” GitHub release, commit 2bd7cd7. [Online]. Available: <https://github.com/NVIDIA/cuvs/releases/tag/v26.06.00>, 2026, accessed: Aug. 25, 2026.
- [7] Zilliz, “VectorDBBench: A benchmark tool for vector databases,” GitHub repository. [Online]. Available: <https://github.com/zilliztech/VectorDBBench>, 2026, accessed: Aug. 25, 2026.
- [8] NVIDIA Corporation, “NVIDIA RTX Blackwell GPU Architecture,” NVIDIA Corporation, Tech. Rep., 2025, [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>. Accessed: Aug. 25, 2026.